Module 2: Decision Making in Java

Java Laboratory – Problem Statements

Module Overview

This module introduces students to Java's decision-making constructs. Students will learn how to control the flow of program execution using conditional statements such as `if`, `if-else`, `nested if`, `else-if ladder`, `switch`, and the ternary operator. The laboratory exercises are designed to strengthen logical thinking and problem-solving skills through real-world scenarios.

Total Practice Problems: 20

Difficulty Distribution:

- Beginner: 8 Problems
- Intermediate: 8 Problems
- Advanced: 4 Problems

Learning Objectives

After completing this module, students will be able to:

- Apply conditional statements to solve decision-making problems.
- Select appropriate conditional constructs for different scenarios.
- Design programs involving multiple conditions.
- Develop menu-driven applications using `switch`.
- Use the ternary operator for concise decision-making.
- Validate user inputs.
- Implement business rules using conditional logic.

Concepts Covered

- `if` Statement
- `if-else`
- Nested `if`
- `else-if` Ladder
- `switch`
- Enhanced `switch` (Java 21)

- Ternary Operator (`?:`)
- Logical Operators
- Relational Operators

Beginner Level Problems

Problem 1: Positive, Negative, or Zero

Difficulty: Beginner

Problem Statement

Accept an integer from the user and determine whether the number is:

- Positive
- Negative
- Zero

Use the `if-else` statement.

Problem 2: Even or Odd Number

Difficulty: Beginner

Problem Statement

Write a Java program to accept an integer and determine whether it is even or odd.

Display an appropriate message.

Problem 3: Largest of Two Numbers

Difficulty: Beginner

Problem Statement

Accept two numbers from the user and determine which one is larger.

If both numbers are equal, display an appropriate message.

Problem 4: Voting Eligibility

Difficulty: Beginner

Problem Statement

Accept a person's age and determine whether they are eligible to vote.

Assume the minimum voting age is 18 years.

Problem 5: Pass or Fail

Difficulty: Beginner

Problem Statement

Accept the marks obtained by a student.

If the marks are 40 or above, display **"Pass"**; otherwise display **"Fail"**.

Problem 6: Leap Year Checker

Difficulty: Beginner

Problem Statement

Accept a year from the user and determine whether it is a leap year.

Problem 7: Divisibility Checker

Difficulty: Beginner

Problem Statement

Accept an integer and determine whether it is divisible by both 5 and 11.

Problem 8: Character Type Identifier

Difficulty: Beginner

Problem Statement

Accept a single character and determine whether it is:

- Uppercase alphabet
- Lowercase alphabet
- Digit
- Special character

Intermediate Level Problems

Problem 9: Grade Calculator

Difficulty: Intermediate

Problem Statement

Accept marks (0–100) and assign grades according to the following criteria:

Marks	Grade
90–100	A+
80–89	A
70–79	B
60–69	C
50–59	D
Below 50	F

Also validate that the entered marks are within the valid range.

Problem 10: Largest of Three Numbers

Difficulty: Intermediate

Problem Statement

Accept three numbers and determine the largest among them using nested `if` statements.

Problem 11: Triangle Validator

Difficulty: Intermediate

Problem Statement

Accept the lengths of three sides and determine whether they can form a valid triangle.

If valid, classify the triangle as:

- Equilateral
- Isosceles
- Scalene

Problem 12: Income Tax Category

Difficulty: Intermediate

Problem Statement

Accept the annual income of an individual and determine the applicable tax slab based on predefined income ranges.

Display only the applicable slab category, not the tax amount.

Problem 13: Electricity Bill Category

Difficulty: Intermediate

Problem Statement

Accept the number of electricity units consumed and classify the consumer into categories such as:

- Domestic
- Commercial
- High Consumption

Use an `else-if` ladder.

Problem 14: Day of the Week

Difficulty: Intermediate

Problem Statement

Accept a number (1–7) and display the corresponding day of the week using a `switch` statement.

If the input is invalid, display an appropriate error message.

Problem 15: Simple Menu-Driven Calculator

Difficulty: Intermediate

Problem Statement

Display the following menu:

1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit

Accept the user's choice and perform the selected operation using a `switch` statement.

Problem 16: Student Scholarship Eligibility

Difficulty: Intermediate

Problem Statement

Determine whether a student is eligible for a scholarship based on the following conditions:

- Percentage is at least 85%.
- Attendance is at least 90%.
- No disciplinary action.

Display whether the student is eligible or not.

Advanced Level Problems

Problem 17: Employee Performance Evaluation

Difficulty: Advanced

Problem Statement

Accept the following inputs:

- Employee ID
- Years of Experience
- Performance Rating (1–5)

Based on these values, classify the employee into one of the following categories:

- Outstanding
- Excellent
- Good
- Needs Improvement

Clearly define and implement suitable decision rules.

Problem 18: Banking Loan Eligibility

Difficulty: Advanced

Problem Statement

Determine the eligibility of a loan applicant based on:

- Age
- Monthly Income
- Employment Status
- Credit Score

Display one of the following outcomes:

- Loan Approved
- Loan Approved with Conditions
- Loan Rejected

Design appropriate decision rules.

Problem 19: Smart Parking Fee Calculator

Difficulty: Advanced

Problem Statement

Develop a parking fee calculator that determines the parking charges based on:

- Vehicle Type
- Parking Duration
- Membership Status

Use nested conditional statements and `switch` where appropriate.

Problem 20: Online Shopping Discount System

Difficulty: Advanced

Problem Statement

Develop a shopping discount calculator.

Accept:

- Customer Type (Regular, Silver, Gold, Platinum)
- Purchase Amount
- Festival Offer (Yes/No)

Calculate the applicable discount according to predefined business rules and display the final payable amount.

Challenge Problems

1. Admission eligibility checker for a university based on multiple criteria.
2. Railway ticket fare calculator considering age, class, and concession.
3. Hospital triage system that categorizes patients based on symptoms and severity.
4. Mobile recharge plan recommender based on usage preferences.
5. Restaurant bill generator with discounts and membership benefits.

Real-World Applications

The concepts learned in this module can be applied to develop:

- ATM transaction systems
- Student grading systems
- Banking loan approval systems
- Employee appraisal systems
- Shopping cart and billing applications
- Admission and scholarship portals
- Parking management systems
- Utility billing systems
- Online examination portals
- Customer support ticket categorization

Common Mistakes

Students should avoid the following errors:

- Using `=` instead of `==` for comparison.
- Omitting braces `{}` in nested conditions, leading to logic errors.
- Incorrect use of logical operators (`&&`, `||`, `!`).
- Not validating user input before processing.
- Missing `break` statements in `switch` (where applicable).
- Writing overlapping or unreachable `else-if` conditions.
- Ignoring boundary values (e.g., exactly 18 years, exactly 40 marks).

- Using multiple independent `if` statements instead of an `if-else` chain when only one condition should execute.

Viva Questions

1. What is the purpose of decision-making statements in Java?
2. Differentiate between `if`, `if-else`, and `else-if` ladder.
3. When should nested `if` statements be used?
4. Explain the syntax and use cases of the `switch` statement.
5. What is the difference between the traditional and enhanced `switch` introduced in modern Java?
6. What is the ternary operator, and when is it preferred?
7. Differentiate between relational and logical operators.
8. Why is input validation important in decision-making programs?
9. What happens if a `break` statement is omitted in a traditional `switch`?
10. How can complex business rules be simplified using decision structures?

Expected Learning Outcomes

Upon successful completion of Module 2, students will be able to:

- Develop Java programs that make decisions based on user input.
- Select the appropriate conditional construct for different scenarios.
- Validate and process inputs using conditional logic.
- Implement real-world business rules in Java applications.
- Design menu-driven programs using `switch`.
- Build a strong foundation for future topics such as loops, methods, and object-oriented programming.